```
      expect(ts.score).to eq("15-0")
    end

    it "raises an error if it doesn't know the player" do
      ts = TennisScorer.new
      expect { ts.give_point_to(:referee) }.to raise_error(RuntimeError)
    end

    it "makes the score 0-15 if the receiver wins a point"
    it "makes the score 15-15 after they both win a point"
  end
end
```

This won't pass yet, because our TennisScorer class doesn't implement a give_point_to method. Let's rectify that. Our code isn't finished, but now the existing specs will pass:

unittesting/bdd/3/tennis_scorer.rb

```
class TennisScorer
  PLAYERS = %i[server receiver]

  def initialize
    @score = {server: 0, receiver: 0}
  end

  def score
    "#{@score[:server] * 15}-#{@score[:receiver] * 15}"
  end

  def give_point_to(player)
    raise "Unknown player #{player}" unless PLAYERS.include?(player)
    @score[player] += 1
  end
end
```

Again, we'll run the file:

```
$ rspec ts_spec.rb
...**
Pending: (Failures listed here are expected and do not affect your suite's status)
  1) TennisScorer basic scoring makes the score 0-15 if the receiver wins a point
     # Not yet implemented
     # ./ts_spec.rb:21
  2) TennisScorer basic scoring makes the score 15-15 after they both win a point
     # Not yet implemented
     # ./ts_spec.rb:22
Finished in 0.00355 seconds (files took 0.04966 seconds to load)
5 examples, 0 failures, 2 pending
```

We're now meeting two of the four initial expectations. But, before we move on, note there's a bit of duplication in the specification: all of our expectations create a new TennisScorer object. We can fix that by using a before method in the specification. This works a bit like the setup method in minitest, allowing us to run code before expectations are executed. Let's use this feature and, at the same time, build out the last two expectations:

unittesting/bdd/4/ts_spec.rb

```
require_relative "tennis_scorer"

RSpec.describe TennisScorer do
  describe "basic scoring" do
```